



Intelligent Agents

SE-350/SE-450, Spring-2026
Artificial Intelligence (AI)

Dr. Kanwal Yousaf

Department of Software Engineering
UET Taxila

e-mail: kanwal.yousaf@uettaxila.edu.pk



Group to join for updates

- All course material and important information will be shared in the MS Teams group below:

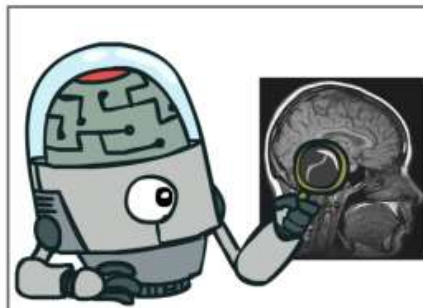
SE-350/SE-450 : Artificial Intelligence (Session-2023/2022)

rbuy693

In simpler terms AI is...

- The science of making machines that:

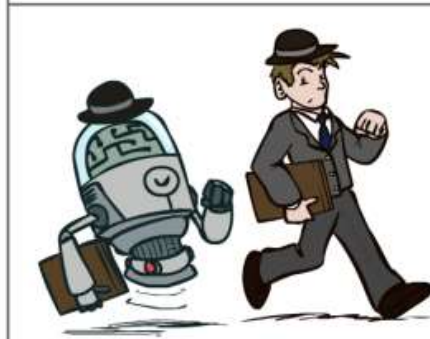
Think like people



Think rationally



Act like people



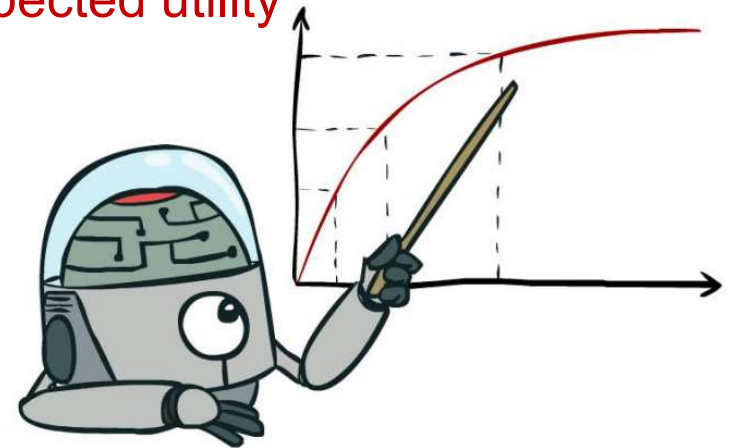
Act rationally



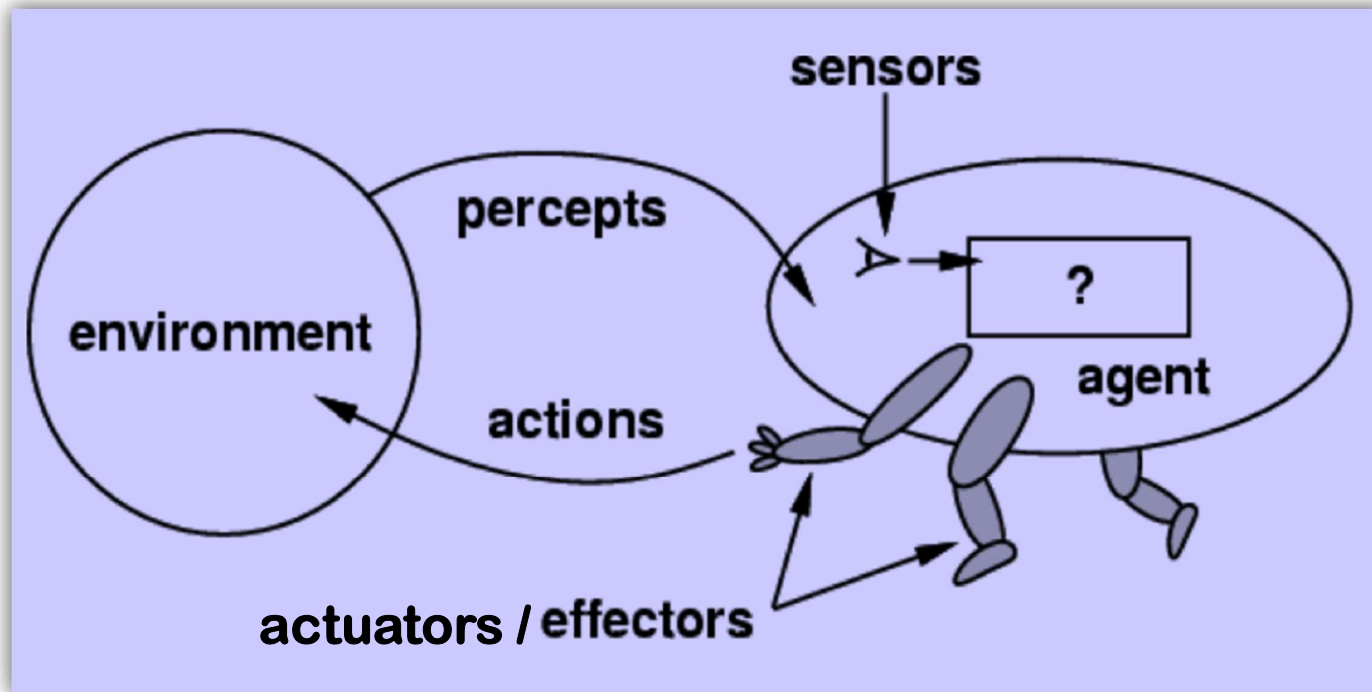
Rational decisions

- Should we make machines that do **rational decisions**...
- **Rational**: *maximally* achieving pre-defined goals
- Goals are expressed in terms of the **utility** of outcomes
- World is uncertain, so we'll use **expected** utility
- **Being rational means acting to maximize your expected utility**

A better title for this course might be:
Introduction to Computational Rationality



Agents & Environments



- **Agents** include humans, robots, softbots, thermostats, etc.
- The **agent function** maps from percept histories to actions:

$$f : \mathcal{P}^* \rightarrow A$$

Vacuum cleaner world

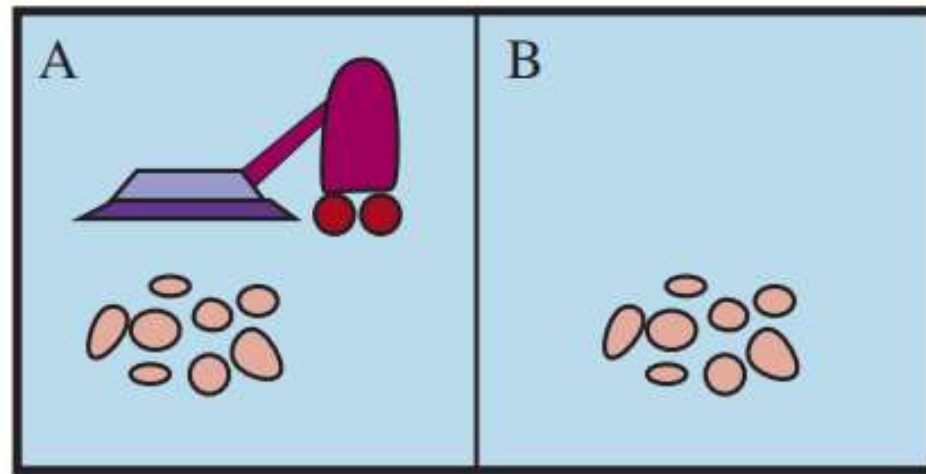


Figure 2.2 A vacuum-cleaner world with just two locations. Each location can be clean or dirty, and the agent can move left or right and can clean the square that it occupies. Different versions of the vacuum world allow for different rules about what the agent can perceive, whether its actions always succeed, and so on.

- Percepts: location and contents, e.g., *[A, Dirty]*
- Actions: *Left, Right, Suck, NoOp*

Vacuum cleaner agent

Table

Percept sequence	Action
$[A, Clean]$	<i>Right</i>
$[A, Dirty]$	<i>Suck</i>
$[B, Clean]$	<i>Left</i>
$[B, Dirty]$	<i>Suck</i>
$[A, Clean], [A, Clean]$	<i>Right</i>
$[A, Clean], [A, Dirty]$	<i>Suck</i>
$\vdots$	$\vdots$

- What is the right function?
- Can it be implemented in a small agent program?

Function

Input: location, status

Output: action

```

1: if status = Dirty then
2:   return Suck
3: end if
4: if location = A then
5:   return Right
6: end if
7: if location = B then
8:   return Left
9: end if

```



Rationality

- Fixed performance measure evaluates the environment sequence
 - one point per square cleaned up in time T ?
 - one point per clean square per time step, minus one per move?
 - penalize for $> k$ dirty squares?
- What is rational at any given time depends on four things:
 - The performance measure that defines the criterion of success.
 - The agent's prior knowledge of the environment.
 - The actions that the agent can perform.
 - The agent's percept sequence to date

A **rational agent** chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date.

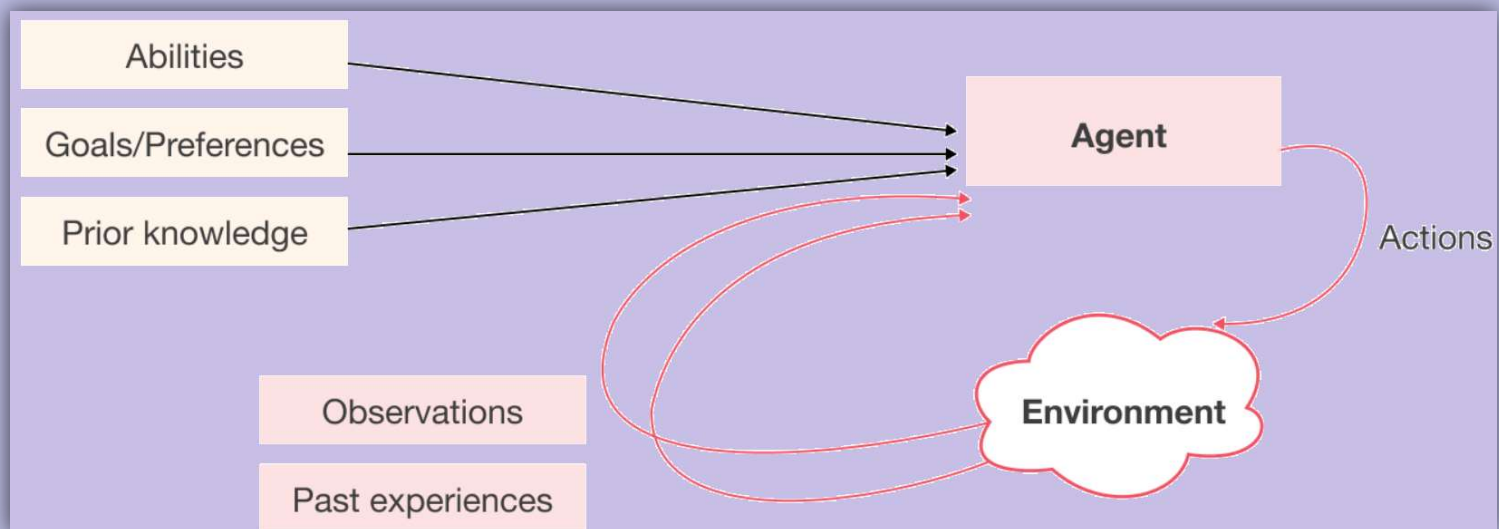


Rationality (cont...)

- **Rational $\neq$ omniscient** (all-knowing)
 - percepts may not supply all relevant information
- **Rational $\neq$ clairvoyant** (predicting the future)
 - action outcomes may not be as expected
- Hence, **rational $\neq$ successful**
- **Rational $\Rightarrow$ exploration, learning, autonomy**

Intelligent agent

An intelligent agent **perceives** its environment via **sensors** and **acts** rationally upon that environment with its **effectors (or actuators)**.





Sensors/Percepts & Effectors (Actuators)/Actions

- For example: **humans**
 - **Sensors:** Eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction), neuromuscular system (proprioception)
 - **Percepts:**
 - At the lowest level: electrical signals from these sensors
 - After preprocessing: objects in the visual field (location, textures, colors, ...), auditory streams (pitch, loudness, direction), ...
 - **Effectors/Actuators:** limbs, digits, eyes, tongue, ...
 - **Actions:** lift a finger, turn left, walk, run, carry an object, ...
- Percepts and actions need to be carefully defined, possibly at different levels of abstraction



Example: self-driving car

- **Percepts:** Video, sonar, speedometer, odometer, engine sensors, keyboard input, microphone, GPS, ...
- **Actions:** Steer, accelerate, brake, horn, speak/display, ...
- **Goals:** Maintain safety, reach destination, maximize profits (fuel, tire wear), obey laws, provide passenger comfort, ...
- **Environment:** U.S. urban streets, freeways, traffic, pedestrians, weather, customers, ...
- Different aspects of driving may require different types of agent programs



Rationality

- An ideal rational agent should, for each possible percept sequence, do whatever actions will maximize its expected performance measure based on
 - percept sequence
 - built-in and acquired knowledge
- Rationality includes information gathering, not "rational ignorance"
(If you don't know something, find out!)
- Need a performance measure to say how well a task has been achieved
- **Types of performance measures**
 - false alarm (false positive) rate
 - false dismissal (false negative) rate
 - speed
 - resources required
 - impact on environment
 - etc.



Agent, agent programs & architectures

- An **agent** is defined by its behavior: the actions it takes in response to percepts.
- The goal of AI is to design an **agent program** that implements the agent function
 - Mapping from percepts $\rightarrow$ actions
- An **agent architecture** is the physical system that runs the program
 - Includes sensors, actuators, and computing hardware
- An agent is defined as:

agent = architecture + program



Agent, agent programs & architectures (cont...)

- A **discrete agent** receives percepts one at a time, and maps this percept sequence to a sequence of **discrete actions**.
- Other possible properties
 - autonomous
 - reactive to the environment
 - pro-active (goal-directed)
 - interacts with other agents via the environment



Agent, agent programs & architectures (cont...)

- The agent program must be **compatible with the architecture**
 - Example: actions like Walk require an architecture with legs

agent = architecture + program

- The architecture:
 - Provides percepts from sensors to the program
 - Executes the program
 - Sends chosen actions to actuators



PEAS

- PEAS: **Performance measure, Environment, Actuators, Sensors**
- Must first specify the setting for intelligent agent design
- Example:
 - **Agent:** Medical diagnosis system
 - **Performance measure:** Healthy patient, minimize costs, lawsuits
 - **Environment:** Patient, hospital, staff
 - **Actuators:** Screen display (questions, tests, diagnoses, treatments, referrals)
 - **Sensors:** Keyboard (entry of symptoms, findings, patient's answers)



Agent types

- **Table-driven agents**

use a percept sequence/action table in memory to find the next action. They are implemented by a (large) lookup table.

- **Simple reflex agents**

are based on condition-action rules, implemented with an appropriate production system. They are stateless devices which do not have memory of past world states.

- **Agents with memory**

have internal state, which is used to keep track of past states of the world.

- **Agents with goals**

are agents that, in addition to state information, have goal information that describes desirable situations. Agents of this kind take future events into consideration..

- **Utility-based agents**

base their decisions on classic axiomatic utility theory in order to act rationally.

Table-driven agents

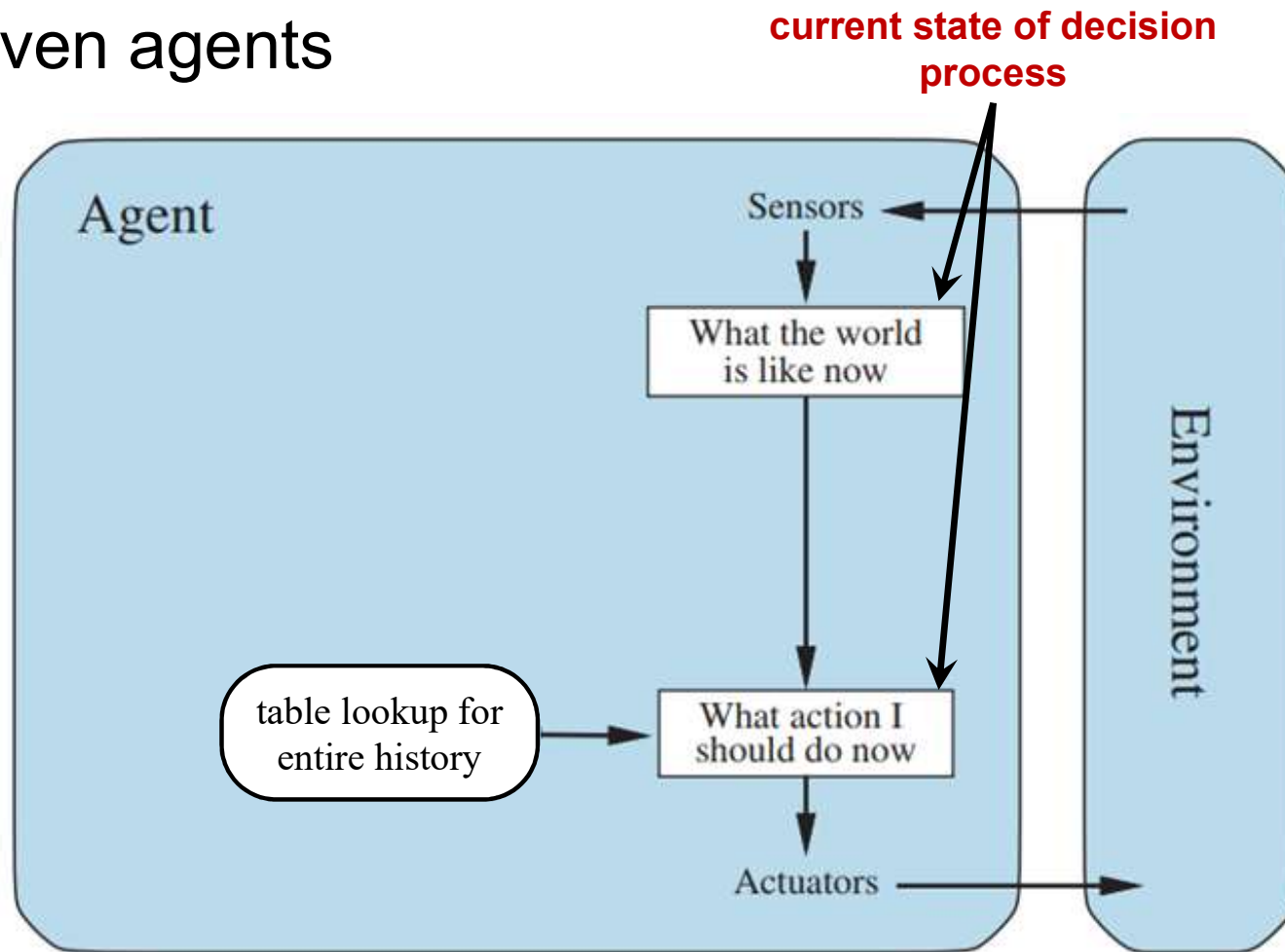


Table-driven agents (cont...)

- Table lookup of percept-action pairs mapping from every possible perceived state to the optimal action for that state

function TABLE-DRIVEN-AGENT(*percept*) **returns** an action

persistent: *percepts*, a sequence, initially empty

table, a table of actions, indexed by percept sequences, initially fully specified

 append *percept* to the end of *percepts*

action $\leftarrow$ LOOKUP(*percepts*, *table*)

return *action*

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

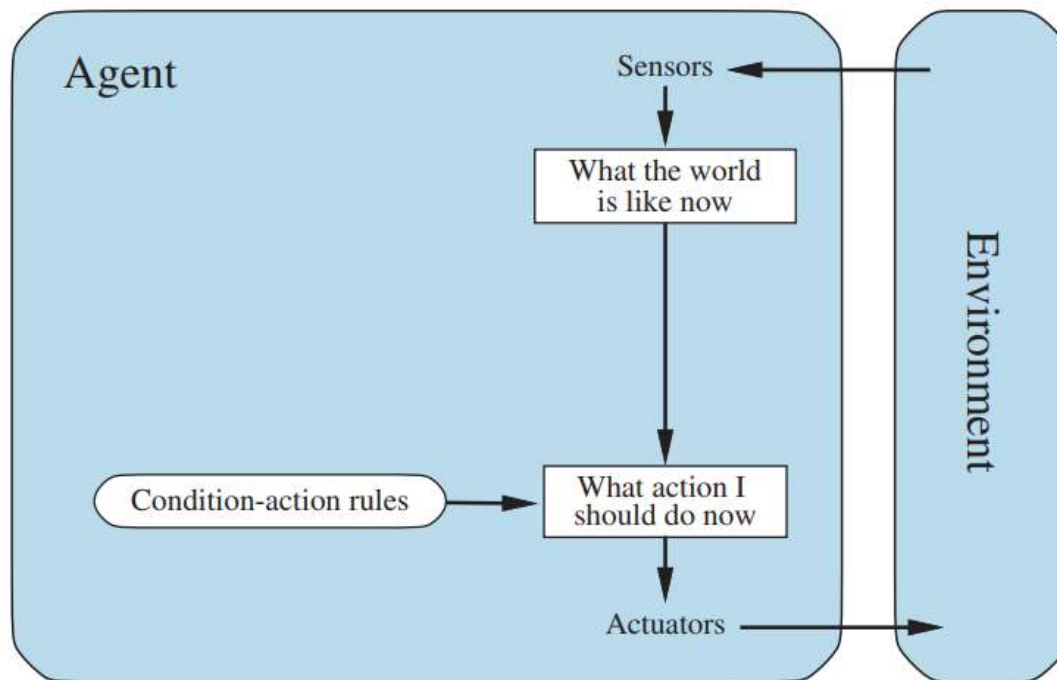


Table-driven agents (cont...)

- **Problems**

- too big to generate and to store (Chess has about 10^{120} states, for example)
- no knowledge of non-perceptual parts of the current state
- not adaptive to changes in the environment; requires entire table to be updated if changes occur
- looping: can't make actions conditional on previous actions/states

Simple reflex agents



function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
persistent: *rules*, a set of condition–action rules

state $\leftarrow$ INTERPRET-INPUT(*percept*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

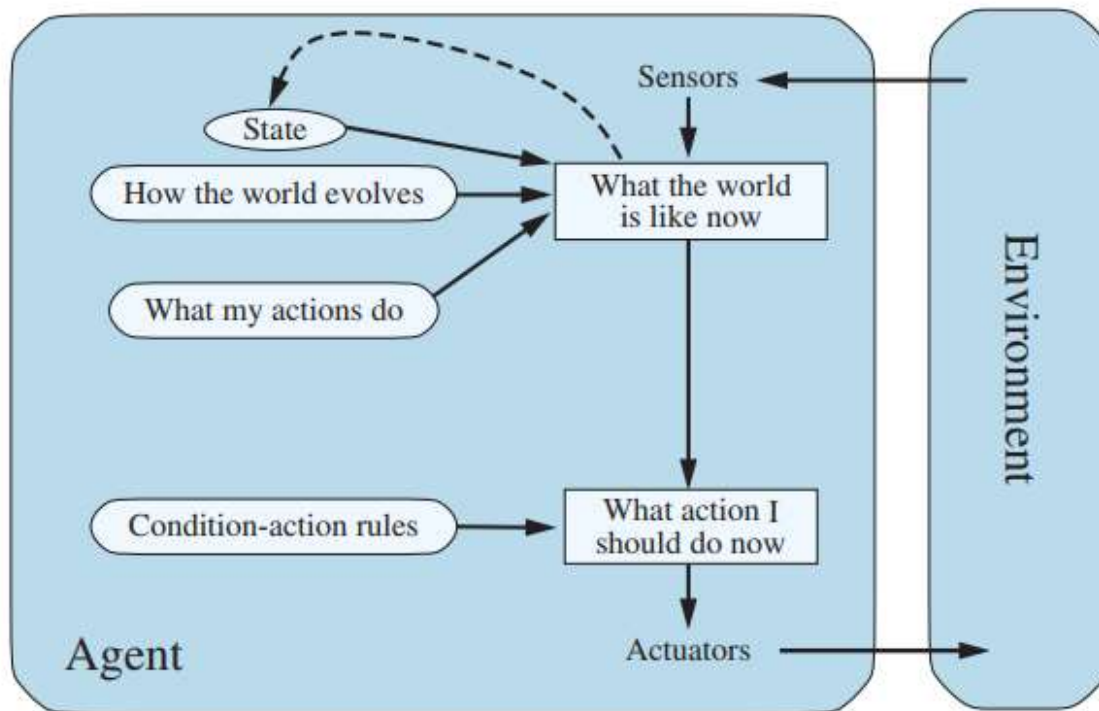
an agent whose actions depends only on the current percepts



Simple reflex agents (cont...)

- **Rule-based reasoning** to map from percepts to optimal action; each rule handles a collection of perceived states
- **Problems**
 - still usually too big to generate and to store
 - still no knowledge of non-perceptual parts of state
 - still not adaptive to changes in the environment; requires collection of rules to be updated if changes occur
 - still can't make actions conditional on previous state

Agents with memory



function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

persistent: *state*, the agent's current conception of the world state
transition_model, a description of how the next state depends on the current state and action
sensor_model, a description of how the current world state is reflected in the agent's percepts
rules, a set of condition-action rules
action, the most recent action, initially none

state ← UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)
rule ← RULE-MATCH(*state*, *rules*)
action ← *rule*.ACTION
return *action*



Agents with memory (cont...)

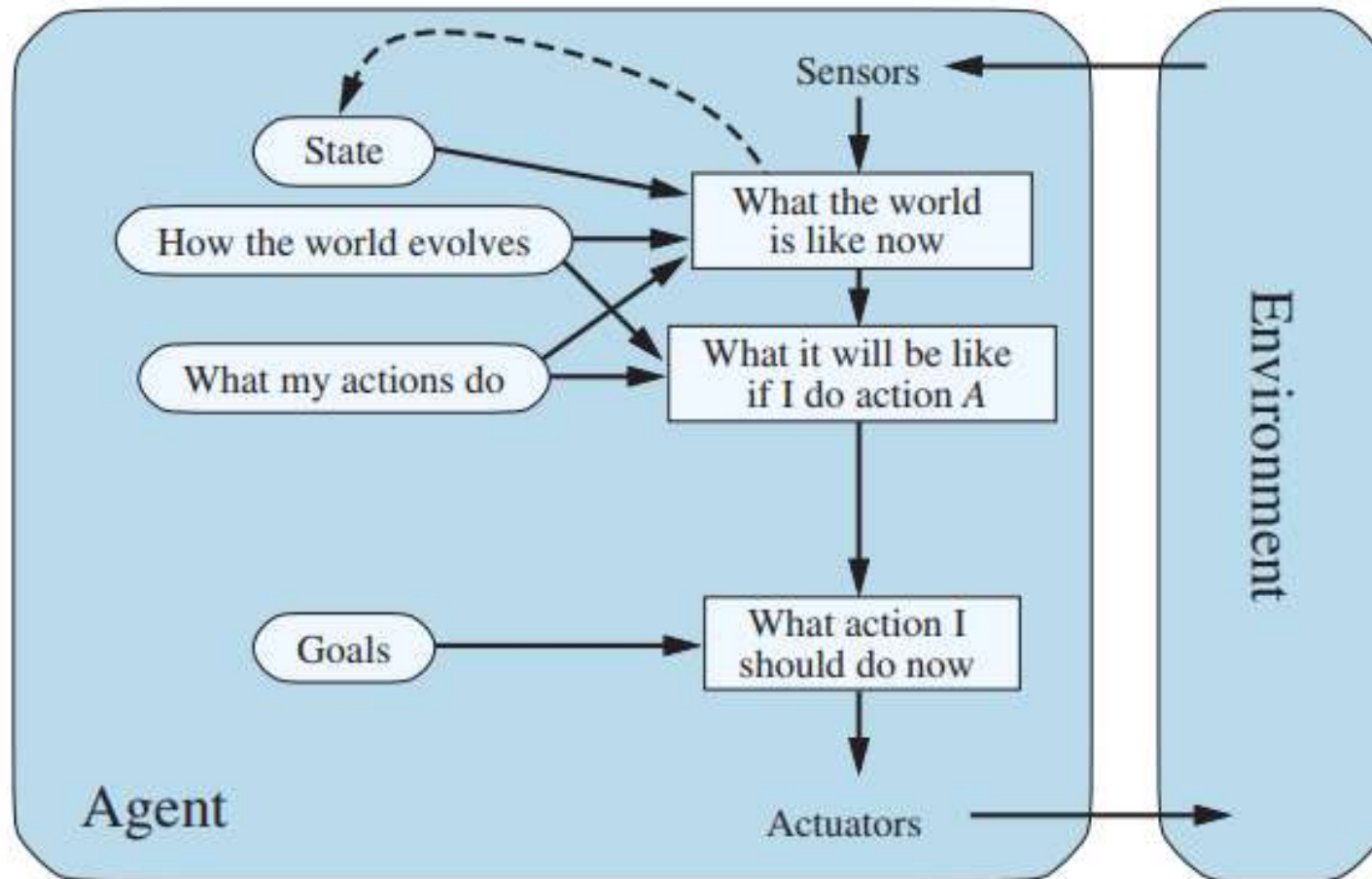
- Encode internal **state** of world to incorporate past percepts
- Needed because sensors do not usually give the entire state of the world at each input, so perception of the environment is captured over time.
- **State** is used to encode different **world states** that generate same percept.
- Example: **Rodney Brooks's subsumption architecture**



Brooks' subsumption architecture

- Main idea: build complex, intelligent robots by **decomposing behaviors** into a hierarchy of skills, each completely defining a complete percept-action cycle for one very specific task
- Examples:
 - wandering
 - avoiding contact
 - follow wall
 - recognizing doorways
- Behaviors are loosely coupled, asynchronous interactions

Goal-based agents

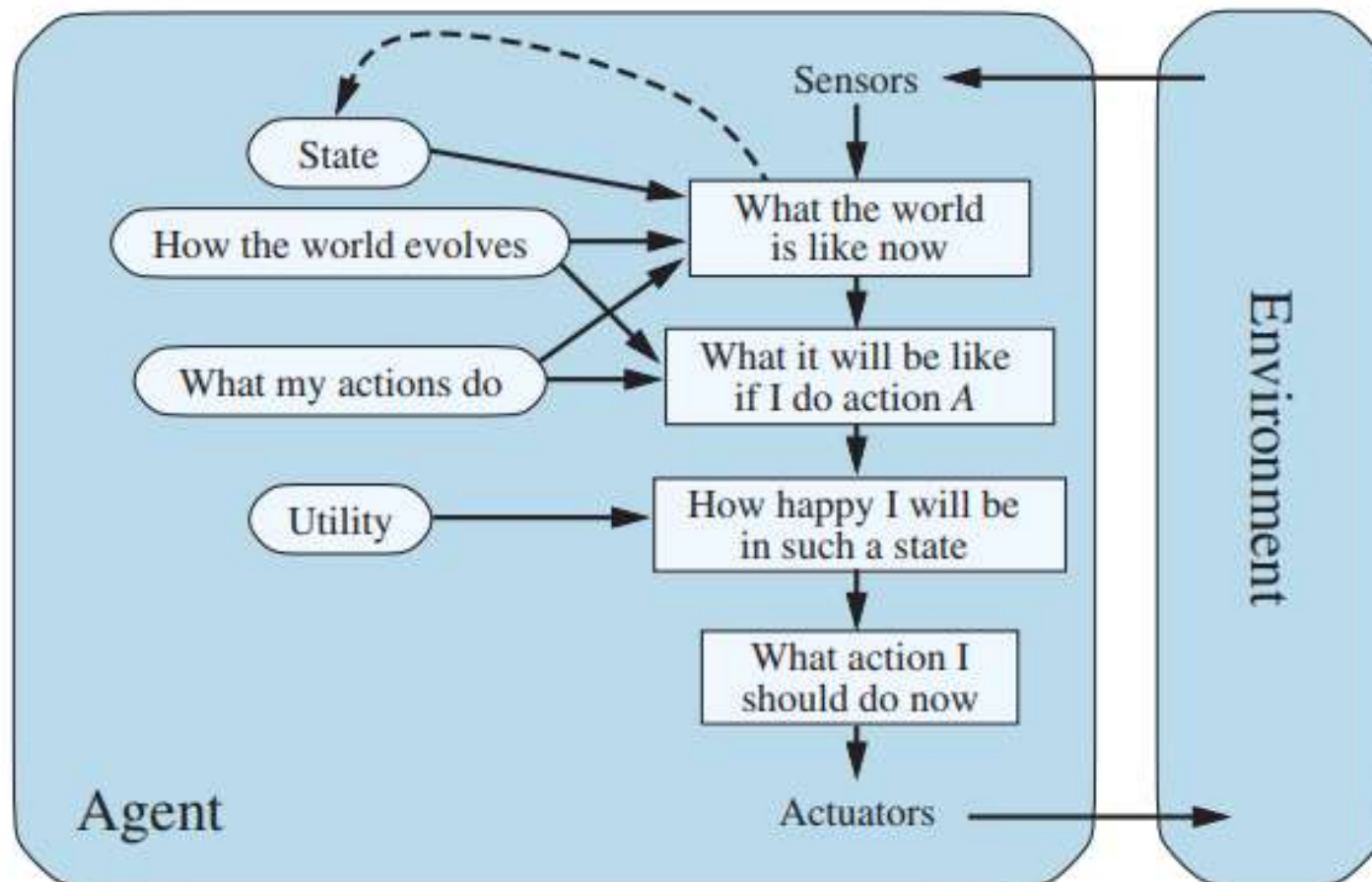




Goal-based agents (cont...)

- Choose actions so as to achieve a (given or computed) goal.
- A goal is a description of a desirable situation.
- Keeping track of the current state is often not enough: need to add goals to decide which situations are good
- **Deliberative** instead of **reactive**.
- May have to consider long sequences of possible actions before deciding if goal is achieved
(involves consideration of the future, "*what will happen if I do...?*")

Utility-based agent





Utility-based agent (cont...)

When there are multiple possible alternatives, how to decide which one is best?

- A goal specifies a crude distinction between a happy and unhappy state, but often need a more general performance measure that describes "degree of happiness."
- **Utility function**

$U: \text{State} \rightarrow \text{Real Numbers}$

indicating a measure of success or happiness when at a given state.

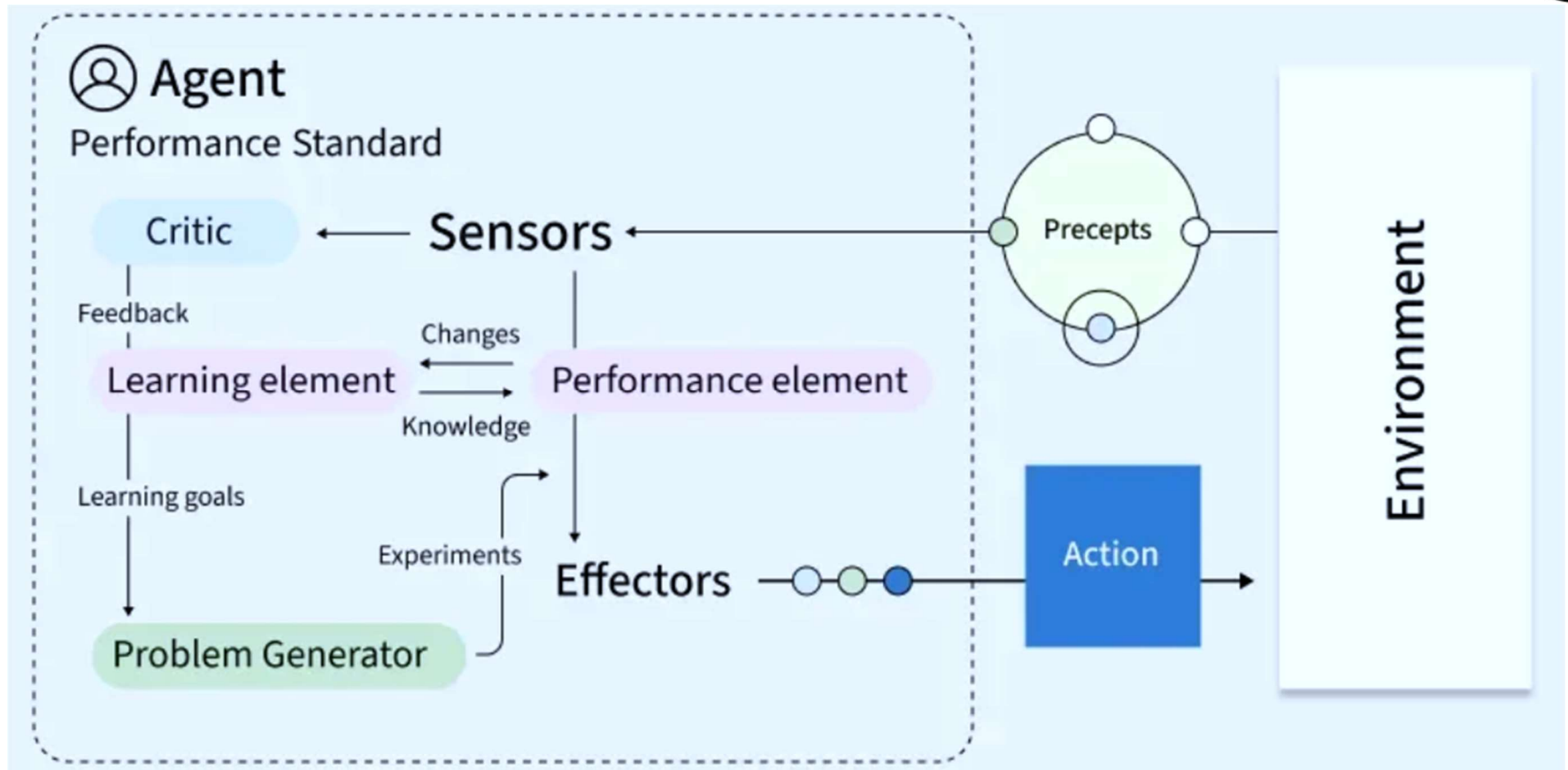
- Allows decisions comparing choice between conflicting goals, and choice between likelihood of success and importance of goal (if achievement is uncertain).



Learning-based agents

- Learning agents improve their performance over time based on experience.
- They modify their behavior by observing the consequences of their actions, adjusting their internal models and decision-making approaches to achieve better outcomes in future interactions.
- **Key Features**
 - Adapt to changing environments
 - Improve performance with experience
 - Contain both a performance element and a learning element
 - Generate new knowledge rather than simply applying existing rules
- **Deepmind's AlphaGo** and other recent breakthroughs use these types of agents.

Learning-based agents (cont...)





Environment properties

- Do the agent's sensors give complete information (relevant to the choice of action) about the state of the environment at each point in time? **Fully vs. partially-observable**
- Does the agent operate in an environment with other agents? **Single vs. multi-agent (competitive, cooperative)**
- Is the next state of the environment completely determined by the current state and agent action? **Episodic vs. sequential**
- Can the environment change while the agent is deliberating? **Static vs dynamic**
- What is the domain of values for variables tracking environment state, agent state, and time? **Discrete vs. continuous**
- Does the agent know outcomes of all its actions? **Known vs unknown**



Environment properties (cont...)

- **Fully observable/Partially observable**

- if an agent's sensors give it access to the complete state of the environment needed to choose an action, the environment is observable (or accessible).
- such environments are convenient, since the agent is freed from the task of keeping track of the changes in the environment.

- **Single-agent / Multiagent**

- If only one agent acts to achieve a goal, the environment is single-agent; if multiple agents interact and affect outcomes, it is multiagent.
- an entity is modeled as an agent only if its behavior is goal-driven and depends on other agents' actions.



Environment properties

- **Competitive/Cooperative**

- an environment is competitive when agents have opposing goals and one agent's success reduces another's performance (e.g., chess players)
- an environment is cooperative when agents share aligned goals and work together to maximize a common performance measure (e.g., taxi-driving environment)

- **Deterministic/Nondeterministic**

- an environment is deterministic if the next state of the environment is completely determined by the current state of the environment and the action of the agent.
- in an accessible and deterministic environment, the agent need not deal with uncertainty.



Environment properties (cont...)

- **Episodic/Sequential**

- an episodic environment means that subsequent episodes do not depend on what actions occurred in previous episodes.
- such environments do not require the agent to plan ahead.

- **Static/Dynamic**

- a static environment does not change while the agent is thinking.
- the passage of time as an agent deliberates is irrelevant.
- the agent doesn't need to observe the world during deliberation.

- **Discrete/Continuous**

- if the number of distinct percepts and actions is limited, the environment is discrete, otherwise it is continuous.



Environment properties (cont...)

- **With/Without intelligent adversaries**

- if the environment contains intelligent, adversarial agents, the agent needs to be concerned about strategic, game-theoretic aspects of the environment.
- most engineering environments don't have rational adversaries, whereas most social and economic systems get their complexity from the interactions of (more or less) rational agents.



Environment properties (cont...)

	Accessible	Deterministic	Episodic	Static	Discrete
Image Classification					
Solitaire					
Backgammon					
Customer support chat					
Taxi driving					



Environment properties (cont...)

	Accessible	Deterministic	Episodic	Static	Discrete
Image Classification	yes	yes	yes	yes	no
Solitaire	no	yes	no	yes	yes
Backgammon	yes	no	no	yes	yes
Customer support chat	no	no	no	no	yes
Taxi driving	no	no	no	no	no

Where do Agentic AI fit?



Agentic AI in the context of Intelligent Agents

- Agentic AI is not a new agent category.
- It is a modern implementation of:
 - Goal-based agents
 - Utility-based agents
 - Learning agents
- Uses large models, memory, tools, and planning
- Strongly aligns with rational agent framework



How LLM-agents work? The 4 Core Pillars

1. Planning (The "Brain")

- **Task Decomposition:** The agent breaks down a complex goal (e.g., "Research and write a report on AI") into smaller, manageable steps.
- **Self-Reflection:** Like the **Critic** (learning agent), the agent looks at its own past actions to correct mistakes or find better paths.

2. Memory (The "State")

- **Short-term Memory:** This is the current "Context Window" (the chat history). It's what the agent is thinking about *right now*.
- **Long-term Memory:** Agents often use a **Vector Database** (External Knowledge) to store and retrieve information they learned days or weeks ago, similar to how learning-based agent diagram shows "Knowledge" storage.



How LLM-agents work? The 4 Core Pillars (cont...)

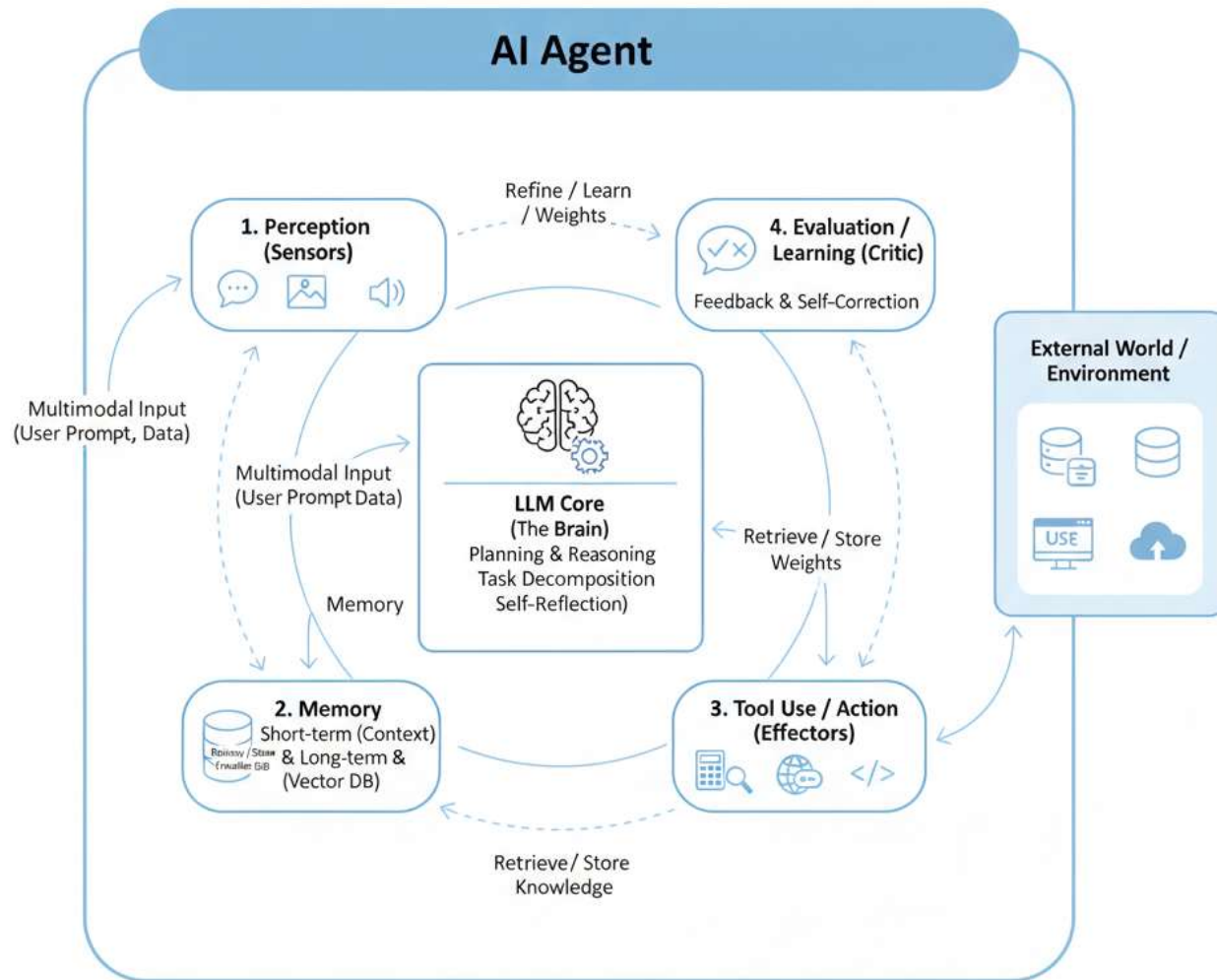
3. Tool Use / Action (The "Effectors")

- For an LLM agent, these are **APIs**. The agent can "call" a calculator to do math, a web search to find news, or a code interpreter to run Python scripts.

4. Perception (The "Sensors")

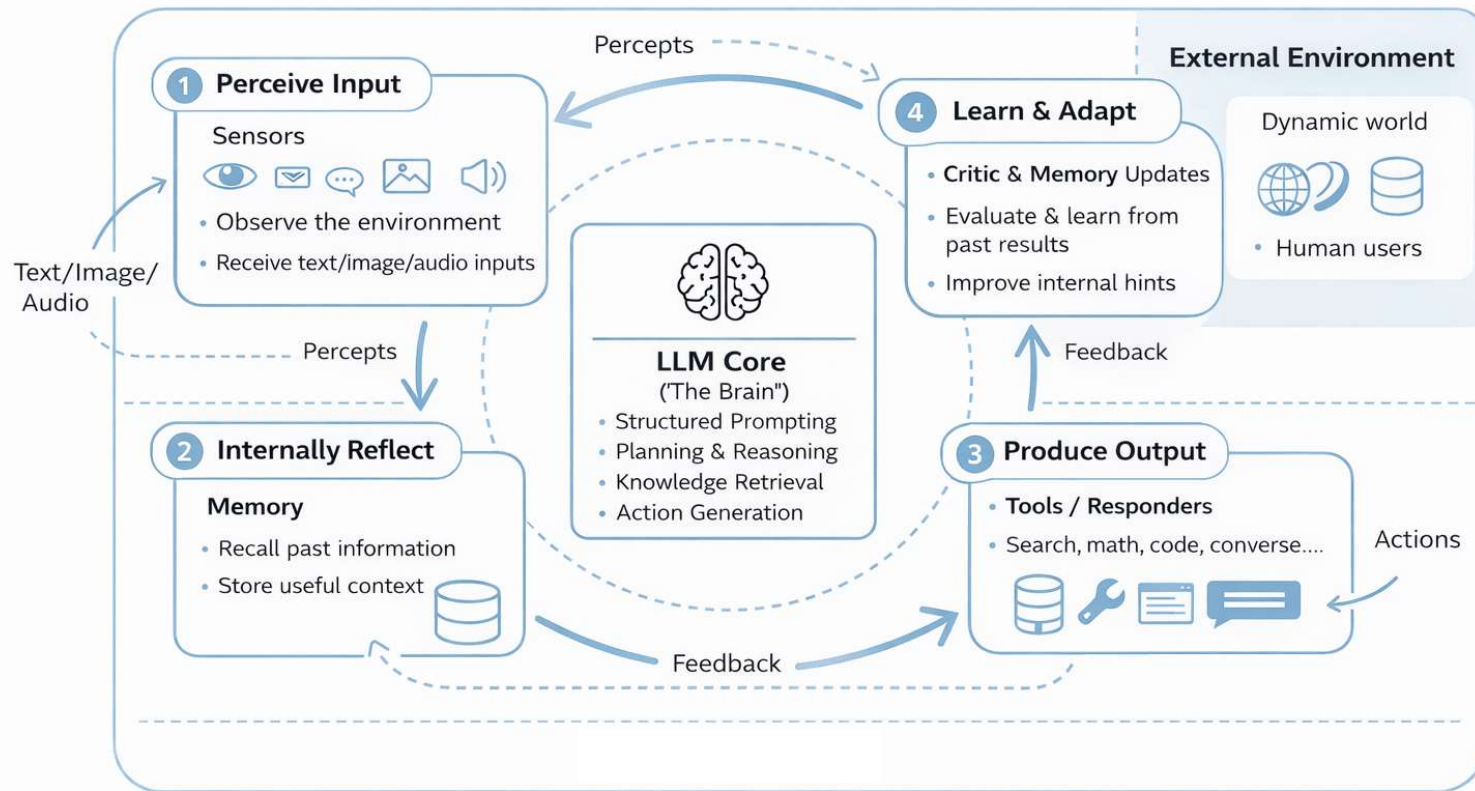
- **Multimodal Input:** Modern agents don't just "read" text. Their sensors can now "see" images, "hear" audio, or read raw data files.

LLM Agent: The ReAct Loop



Agentic LLM: The ReAct Loop

A structured architecture for intelligent agents based on Large Language Models. (Inspired by Russell & Norvig agent model)





Topics Covered

- *Artificial Intelligence: A Modern Approach (4th Edition) by Stuart Russell and Peter Norvig. (Chapter -02)*

Any Question?